

CLIMATE FINANCE VERIFICATION PLATFORM.

1.PROBLEM STATEMENT.

Kenya requires significant climate financing to address challenges such as drought, water scarcity, floods, and environmental degradation. However, communities often lack a transparent and reliable way to track, verify, and report how climate funds are being used at the local project level. This creates opportunities for delays, misuse of funds, incomplete projects, and weak accountability.

Farmers and community members may know whether a project is actually progressing, but their feedback is often disconnected from the systems used by county officials and donors.

MajiShwari addresses this gap by providing a simple SMS-based community reporting and verification system that connects communities, county officials, and donors, while using automated verification and blockchain records to improve transparency and accountability in climate finance - projects.

2. OBJECTIVES.

- 1.To enable community members to report project progress easily through SMS.
- 2.To automatically verify project reports using multiple independent community submissions.
- 3.To provide real-time dashboards for county officials and donors to monitor project status and fund utilization.
- 4.To detect and flag potential fraud or project problems through automated analysis of community reports.
- 5.To create transparent and tamper-resistant records of project verification and fund disbursement using blockchain technology.
- 6.To support informed decision-making by identifying vulnerable areas through data analysis and geographic heatmaps.
- 7.To improve accountability and transparency between communities, government agencies, and climate-finance donors.

3. Methodology

The development of MajiShwari will follow the Software Development Life Cycle (SDLC) using an iterative and prototype-based approach.

1. Requirements Analysis

The system requirements will be identified based on the needs of the main users: farmers, community elders, county officers, and donors. The key requirements will include SMS reporting, project verification, fund tracking, dashboards, fraud detection, and transparent record keeping.

2. System Design

The system architecture and user interfaces will be designed. The platform will consist of:

SMS interface for community reporting.

FastAPI backend for processing reports.

Supabase/PostgreSQL for storing project and verification data.

React dashboard for monitoring projects.

Blockchain smart contract for recording approved fund releases.

Map-based visualization for displaying project and vulnerability information.

3. System Development

The platform will be implemented using:

Python/FastAPI for the backend.

Twilio for SMS communication.

React, TypeScript and Tailwind CSS for the dashboard.

Supabase/PostgreSQL for data storage.

Solidity and Polygon for blockchain functionality.

Python-based data analysis for vulnerability scoring.

4. Verification Method

Community members will submit project reports using a predefined SMS format, for example:

BOREHOLE123 DONE

The system will:

Validate the message.

Identify the project.

Check for duplicate submissions.

Record the report.

Count unique community confirmations.

Mark the project Verified once the required threshold is reached.

Reports indicating problems will automatically trigger an audit flag.

5. Testing

The system will be tested at different levels:

Unit testing – individual functions.

Integration testing – SMS, database, API and blockchain interaction.

End-to-end testing – complete farmer-to-fund-release workflow.

Performance testing – response time and system capacity.

User acceptance testing – testing usability with representative users.

6. Deployment

The completed system will be deployed using cloud services. The frontend will be hosted on Vercel, the backend on a cloud server, the database on Supabase, and the smart contract on a Polygon network.

7. Evaluation

The platform will be evaluated based on:

SMS response time.

Accuracy of project verification.

Detection of duplicate/fraudulent reports.

Dashboard performance.

Reliability of fund-release records.

User experience and system accessibility.

Overall methodology:

Requirements → Design → Development → Testing → Deployment →

Evaluation → Improvement.